

Universidad CENFOTEC
Escuela de Software

Primer estudio de caso

Aplicación de las pilas al análisis de cadenas de impresión

Curso: SOFT-10, Estructuras de Datos
Sección: SCV5 | Periodo: C3-2026
Docente facilitador: Romario Salas Cerdas

Estudiante: Braden Lee Knuter Cordoba
Carné: _____

Fecha de entrega: 11 de octubre del 2026

Repositorio del proyecto en GitHub:

<https://github.com/B0W3YYY/CasoDeEstudio1-EstructuraDeDatos>

1. Introducción y alternativa seleccionada

De las dos aplicaciones clásicas propuestas en la consigna, este trabajo desarrolla el **análisis de cadenas de impresión**. El analizador construido recibe una expresión como las que se escriben dentro de una instrucción de impresión y determina si está bien formada. Los tokens que contempla son exactamente los tres que pide el enunciado: cadenas literales delimitadas por comillas dobles, variables de cadena identificadas por su nombre y el operador de concatenación `+`. A ellos se suman los paréntesis de agrupación, que son los que obligan a recordar un contexto y, por lo tanto, los que justifican el uso de una pila.

Un ejemplo de expresión válida es "Hola, " + nombre + ("!" + salto), mientras que ("Total: " + monto es inválida porque el paréntesis que se abrió nunca se cerró. El problema consiste, entonces, en decidir si cada delimitador de apertura encuentra su delimitador de cierre y si los operandos y operadores se alternan correctamente. Toda la solución, incluida la pila, se programó en Java sin recurrir a las clases de colección del lenguaje, de manera que la estructura de datos queda implementada explícitamente mediante nodos enlazados.

2. Por qué una pila es la estructura de datos adecuada

El requerimiento central del análisis de cadenas de impresión es verificar el **correcto anidamiento** de los delimitadores. Esta propiedad tiene una característica que determina la estructura de datos: cuando se encuentra un delimitador de cierre, el único delimitador de apertura con el que puede emparejarse es el **más recientemente abierto y todavía pendiente**. En la expresión ("a" + (b + "c")), el primer paréntesis de cierre no corresponde al paréntesis inicial, sino al segundo, porque es el último que quedó abierto. Ese comportamiento, en el que el último elemento que entra es el primero que sale, es precisamente la política LIFO (*Last In, First Out*) que define a una pila.

La correspondencia no es una analogía sino una equivalencia formal. El lenguaje de los paréntesis balanceados no es un lenguaje regular: no puede reconocerse con un autómata de estados finitos, porque la profundidad de anidamiento es ilimitada y un autómata finito no puede llevar una cuenta arbitraria. Se trata de un lenguaje independiente del contexto, y el modelo de máquina que reconoce esa clase de lenguajes es exactamente el autómata de pila (Aho, Lam, Sethi y Ullman, 2007).

Dicho de otro modo, la pila no es una opción cómoda entre varias: es la memoria mínima que el problema exige.

Conviene además descartar las alternativas. Un simple contador de paréntesis abiertos detectaría que faltan cierres, pero no podría distinguir entre tipos de delimitadores ni informar en qué posición quedó abierto el que nunca se cerró; tampoco serviría si se combinan comillas y paréntesis. Una lista o un arreglo recorridos libremente permitirían resolver el problema, pero ofrecerían operaciones de acceso que el algoritmo nunca necesita y que, si se usaran por error, producirían emparejamientos incorrectos. Una cola, con su política FIFO, emparejaría el cierre con el delimitador *más antiguo*, que es justamente lo contrario de lo que el anidamiento requiere. La pila expone solo las tres operaciones que el problema necesita —insertar en la cima, retirar de la cima y consultar la cima— y esa restricción es la que vuelve correcto el algoritmo.

3. Orden, seguridad y eficiencia que aporta la pila

3.1 Orden

La pila impone el orden de resolución sin que el programador tenga que administrarlo. Al recorrer la expresión de izquierda a derecha, cada apertura se apila y cada cierre consume la cima; el resultado es que los delimitadores se resuelven en el orden inverso al de su aparición, que es el orden correcto del anidamiento. El estado de la pila en cualquier instante describe con exactitud el contexto en el que se está leyendo: su tamaño es la profundidad actual de anidamiento y su contenido, de la cima a la base, es la lista de contextos abiertos. Esa información ordenada es la que permite emitir un diagnóstico preciso en lugar de un simple aviso de error.

3.2 Seguridad

La seguridad que aporta la pila opera en dos niveles. En el nivel del algoritmo, la interfaz reducida de la estructura hace que ciertos errores sean imposibles de cometer: como no existe un método para leer o modificar un elemento intermedio, no hay forma de emparejar un cierre con un delimitador que no sea el correcto. En el nivel del programa, la pila implementada en este proyecto está diseñada para no interrumpir la ejecución ante un uso indebido: los métodos `pop()` y `peek()` devuelven `null` cuando la pila está vacía en lugar de lanzar una excepción, de modo que quien

los invoca comprueba el resultado y muestra un mensaje al usuario. Gracias a ello, una entrada mal formada como "Hola" + nombre) produce un mensaje explicativo y no una terminación abrupta del programa.

Este aspecto tiene una lectura más amplia. El análisis de delimitadores es una forma de validación de entrada, y validar la estructura de una cadena antes de procesarla es una práctica de seguridad habitual: una cadena de impresión con comillas sin cerrar puede alterar el significado del texto que la sigue, problema emparentado con las inyecciones de código que aparecen cuando se construyen instrucciones concatenando texto sin verificar.

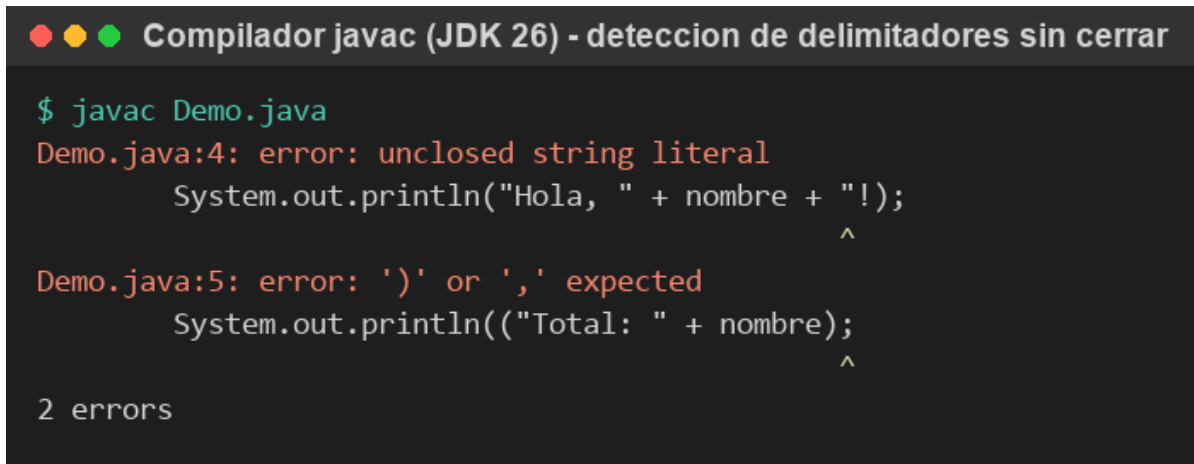
3.3 Eficiencia

Las tres operaciones de la pila trabajan únicamente sobre la cima y no recorren la estructura, por lo que su costo es constante, $O(1)$. Como el analizador ejecuta un número acotado de operaciones por cada carácter leído y cada carácter se visita una sola vez, el análisis completo de una expresión de longitud n se resuelve en tiempo $O(n)$, es decir, en una sola pasada. El consumo de memoria no es proporcional a la longitud de la expresión sino a su profundidad máxima de anidamiento, que en una cadena de impresión real es muy pequeña: solo se almacenan los delimitadores que permanecen abiertos. Al estar construida con nodos enlazados, la pila crece y decrece sin reservar espacio por anticipado ni copiar elementos.

4. Ejemplo concreto en software de sistema

El caso más directo de esta funcionalidad en software de sistema es el propio **compilador de Java, javac**, incluido en el JDK. Antes de traducir un programa, el compilador debe dividir el texto fuente en tokens y verificar que estén bien formados; entre esas comprobaciones están precisamente las dos que realiza el analizador de este proyecto: que toda cadena literal se cierre y que los paréntesis de una instrucción de impresión estén balanceados. La especificación del lenguaje establece en su sección 3.10.5 que un literal de cadena está delimitado por comillas dobles y que constituye un error de compilación que aparezca un terminador de línea entre la comilla de apertura y la de cierre, lo que convierte a una comilla sin cerrar en un error detectable (Oracle, 2023). El análisis sintáctico que resuelve el anidamiento de paréntesis se apoya, como describe la literatura de compiladores, en un autómata de pila (Aho et al., 2007).

Para ilustrar el comportamiento se compiló un archivo que contiene deliberadamente los dos errores. La figura 1 muestra la salida real de javac:



```
●●● Compilador javac (JDK 26) - deteccion de delimitadores sin cerrar

$ javac Demo.java
Demo.java:4: error: unclosed string literal
    System.out.println("Hola, " + nombre + "!);
                                   ^
Demo.java:5: error: ')' or ',' expected
    System.out.println(("Total: " + nombre);
                                   ^
2 errors
```

Figura 1. Salida real del compilador javac del JDK 26 al procesar un archivo con una cadena literal sin cerrar y un paréntesis sin cerrar. El compilador señala la línea y marca con el símbolo ^ la posición exacta del problema.

El resultado es revelador: javac no se limita a indicar que el archivo es incorrecto, sino que identifica el tipo de error (*unclosed string literal*, *)' expected*) y la posición exacta. Esa precisión es posible porque el compilador conserva los contextos abiertos; cuando termina de leer sin haberlos cerrado, sabe cuáles quedaron pendientes. El programa desarrollado en este estudio de caso reproduce ese mismo comportamiento sobre las dos expresiones equivalentes, como se observa en la figura 2, donde además se muestra el estado de la pila después de cada operación.

●●● AnalizadorCadenas.java - mismo diagnostico con la pila propia

```
$ java -cp out Main
Analizando: "Hola, " + nombre + "!
  push '"' (posicion 0) -> [cima] " [base]
  pop '"' (posicion 7) -> (pila vacia)
  push '"' (posicion 20) -> [cima] " [base]
ERROR en la posicion 20: la comilla de apertura no fue cerrada.
Analizando: ("Total: " + nombre
  push '(' (posicion 0) -> [cima] ( [base]
  push '"' (posicion 1) -> [cima] " ( [base]
  pop '"' (posicion 9) -> [cima] ( [base]
ERROR en la posicion 0: el delimitador '(' no fue cerrado.
Programa finalizado. Hasta pronto.
```

Figura 2. Salida real del analizador desarrollado (opción 2 del menú) sobre los mismos dos casos defectuosos. La traza muestra cada push y cada pop junto con el contenido de la pila, y el diagnóstico final indica la posición del delimitador sin cerrar.

La comparación entre ambas figuras permite ver que la lógica es la misma: el delimitador que se abre se recuerda, y la falta de su cierre se detecta porque al terminar el recorrido todavía está pendiente. La diferencia está en la escala, no en el principio. El mismo mecanismo aparece en los editores de código, que resaltan el paréntesis correspondiente al que tiene el cursor, y en intérpretes interactivos que mantienen abierta la entrada mientras quede un delimitador sin cerrar.

5. Representación gráfica de las operaciones

Las figuras siguientes representan la pila como una columna de casillas, donde cada casilla corresponde a un nodo de la clase `Pila` y la casilla superior ocupada es la cima. La figura 3 muestra las operaciones fundamentales sobre la pila de delimitadores.

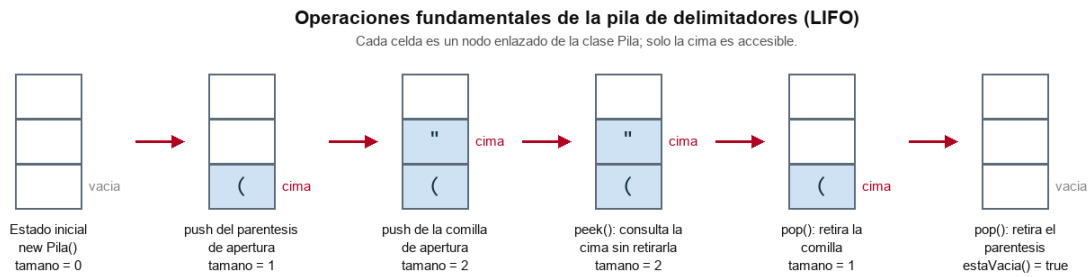


Figura 3. Operaciones fundamentales de la pila: estado inicial, dos operaciones push, una consulta peek que no modifica la estructura y dos operaciones pop que la devuelven al estado vacío.

La figura 4 presenta la traza completa del análisis de una expresión válida. Los pasos y las posiciones indicadas coinciden exactamente con la salida que produce la opción 2 del menú del programa.

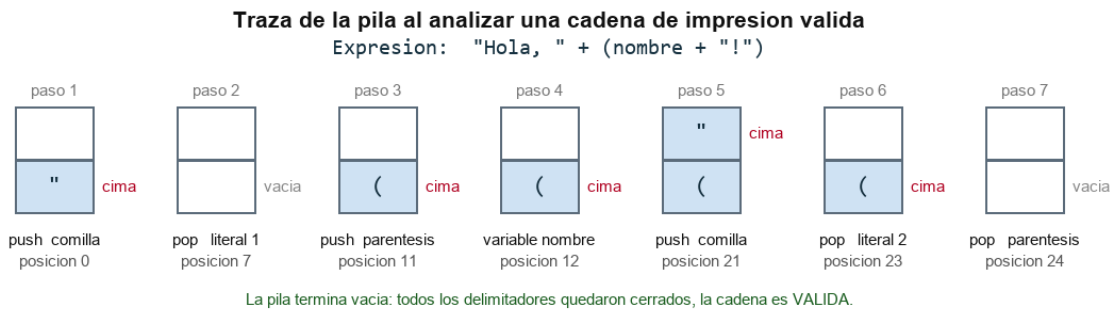
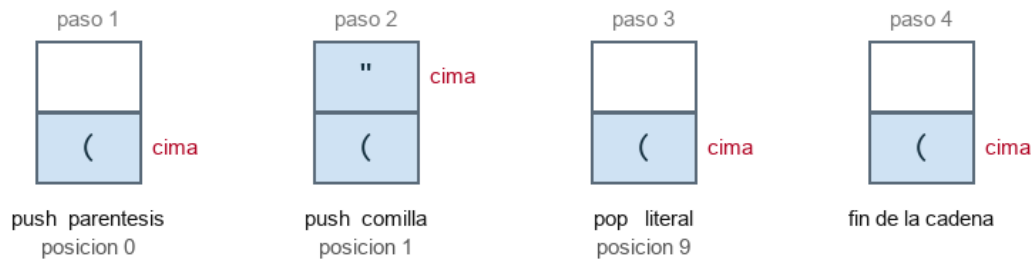


Figura 4. Traza del análisis de la expresión "Hola, " + (nombre + "!"). La pila termina vacía, lo que confirma que todos los delimitadores fueron cerrados.

La figura 5 corresponde al caso inválido. Aquí la pila no queda vacía al terminar el recorrido, y esa condición es la que permite al programa informar cuál delimitador quedó abierto y en qué posición.

Deteccion de un delimitador sin cerrar

Expresion: ("Total: " + nombre



`peek()` informa que el `'('` abierto en la posicion 0 nunca se cerro: la cadena es **INVALIDA**.

Figura 5. Análisis de la expresión inválida `("Total: " + nombre`. Al finalizar, el paréntesis de la posición 0 continúa en la pila, por lo que la cadena se rechaza.

Finalmente, la figura 6 resume el flujo de control del método que realiza el análisis, incluyendo las operaciones adicionales que la funcionalidad requiere además de las fundamentales: la clasificación de cada carácter y la verificación de que operandos y operadores se alternen correctamente.

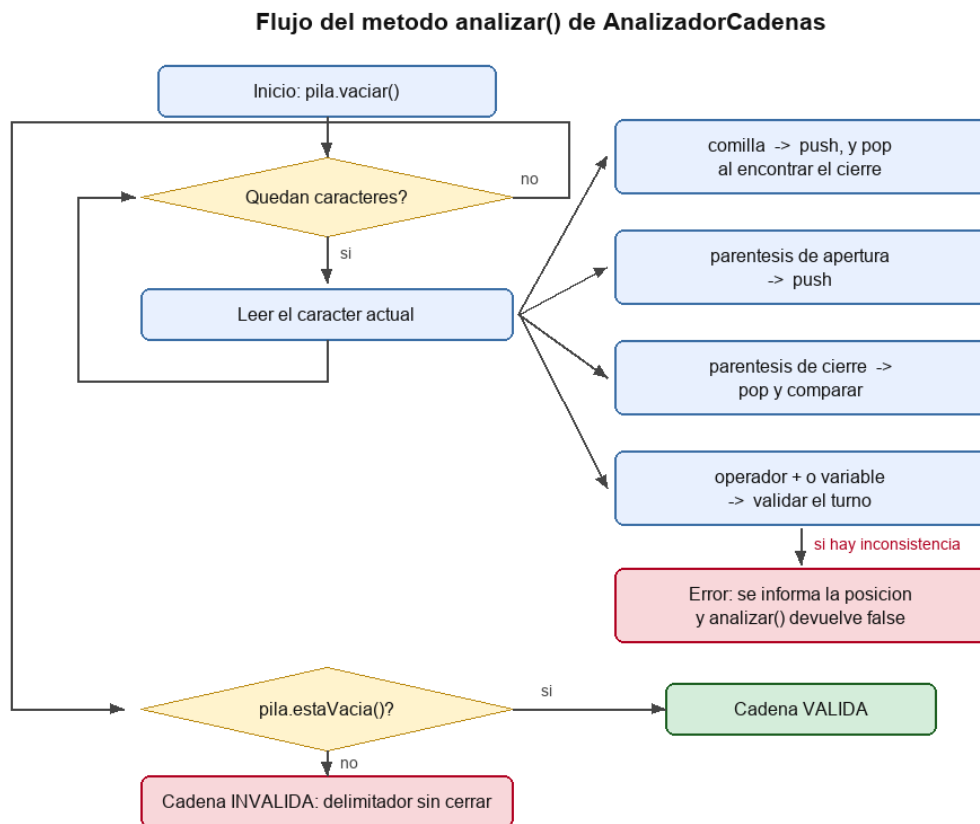


Figura 6. Flujo del método analizar(): recorrido carácter por carácter, clasificación del token, operaciones sobre la pila y verificación final del estado de la pila.

6. Implementación en Java

El proyecto está formado por cuatro clases, cada una con una responsabilidad delimitada. El código fuente completo se encuentra en el repositorio de GitHub indicado al final de este documento y en la portada; aquí se describe únicamente su organización.

- **Nodo.** Representa un elemento de la pila. Tal como indica la consigna para la alternativa seleccionada, el dato almacenado es un carácter que forma parte de la cadena por analizar. Además guarda la posición que ese carácter ocupa dentro de la expresión, dato que permite informar el lugar exacto de un delimitador sin cerrar, y una referencia al nodo situado debajo en la pila.
- **Pila.** Implementa la estructura mediante nodos enlazados, sin usar clases de colección del lenguaje. Ofrece push, pop, peek, estaVacia, getTamano, vaciar, contenido y mostrar; estas dos últimas permiten visualizar el estado de la

estructura.

- **AnalizadorCadenas.** Contiene la lógica del análisis en el método `analizar`, que recorre la expresión una sola vez, usa la pila para los delimitadores, cuenta literales, variables y operadores, y puede emitir la traza paso a paso.
- **Main.** Es el entorno de ejecución. Su método `main()` invoca a `menu()`, que ofrece cinco opciones: analizar una cadena, analizarla mostrando la traza de la pila, ejecutar un conjunto de ejemplos de prueba, operar la pila manualmente mediante un submenú de push, pop y peek, y salir. Las entradas no numéricas o fuera de rango se rechazan con un mensaje y no interrumpen el programa.

7. Pruebas realizadas

El proyecto se compiló con el JDK 26 y se ejecutó probando todas las opciones del menú. Se verificaron expresiones válidas con paréntesis anidados, con una sola variable y con una cadena vacía, y expresiones inválidas de cada tipo previsto: comilla sin cerrar, paréntesis sin cerrar, paréntesis de cierre sin apertura, operador + sin operando a la izquierda o a la derecha, dos operandos consecutivos sin operador y caracteres no permitidos. En todos los casos el mensaje emitido correspondió al error presente y se indicó la posición correcta. También se probó el submenú de operaciones manuales aplicando pop y peek sobre una pila vacía, confirmando que el programa informa la situación y continúa en ejecución. Durante estas pruebas se detectó y corrigió un defecto: cuando la entrada estándar se agotaba, el menú entraba en un ciclo infinito; el programa ahora comprueba si queda entrada disponible y finaliza de forma ordenada.

8. Conclusiones y recomendaciones

El análisis de cadenas de impresión confirma que la elección de la estructura de datos no es un detalle de implementación sino una decisión que determina si el algoritmo puede ser correcto. El anidamiento de delimitadores exige recordar contextos y resolverlos en orden inverso al de su apertura, y la pila es la estructura que realiza esa política de forma natural y con costo constante por operación. La implementación resultante recorre la entrada una sola vez, utiliza memoria proporcional a la profundidad de anidamiento y produce diagnósticos precisos, tal como lo hace un compilador real.

Como recomendaciones para una extensión futura del trabajo se plantean tres. Primera, admitir secuencias de escape dentro de las cadenas literales, de modo que una comilla precedida por una barra invertida no se interprete como cierre. Segunda, reportar todos los errores de una expresión en lugar de detenerse en el primero, mediante una recuperación similar a la que emplean los compiladores. Tercera, reutilizar la misma pila para evaluar la concatenación una vez validada la expresión, lo que acercaría el proyecto al funcionamiento completo de un intérprete.

9. Repositorio del proyecto

El código fuente completo y ejecutable del proyecto de Java se encuentra disponible en el siguiente repositorio público de GitHub:

<https://github.com/B0W3YYY/CasoDeEstudio1-EstructuraDeDatos>

El repositorio contiene la carpeta `src` con los archivos `Nodo.java`, `Pila.java`, `AnalizadorCadenas.java` y `Main.java`, junto con un archivo `README.md` que explica cómo compilar y ejecutar el programa.

Para compilar y ejecutar:

```
javac -d out src/*.java
java -cp out Main
```

10. Referencias

Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2007). *Compilers: Principles, Techniques, and Tools* (2.^a ed.). Pearson/Addison-Wesley.

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4.^a ed.). The MIT Press.

Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2014). *Data Structures and Algorithms in Java* (6.^a ed.). John Wiley & Sons.

Oracle. (2023). *The Java Language Specification, Java SE 21 Edition*. Sección 3.10.5, String Literals.
<https://docs.oracle.com/javase/specs/jls/se21/html/jls-3.html>

Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4.^a ed.). Addison-Wesley.